Magic Attention

A Composable Framework for Exploring Warp-Specialized Attention on Blackwell

Speaker: Manish Gupta

Magic

Contributors & Acknowledgements

Technical Contributors	Manish Gupta[1], Sehban Omer[1], Miles Van Tongeren[1] Daniel Stokes[2], Sergey Klevtsov[2], Pradeep Ramani[2]
Partnership Coordinators	Eric Steinberger[1], Daniel Forester[1] Vartika Singh[2], Morgan Mazouchi[2], Disha Mehra[2], Jyothi Achar[2]
Technical References	FlashAttention, FA1-4, CUTLASS example 77
Infrastructure & Tools	CUTLASS/CuTe, NVCC, PTXAS, NCU, CompileIQ

1 Magic 2 NVIDIA

Magic-Attention

- **Develop** a composable framework of components for attention kernels on NVIDIA Blackwell
- **Analyze** trade-offs with different warp-specialization schedules, Tensor Memory partitioning, and instruction mix
- **Cover** functionality & performance, over a wide range of training, inference, and RL workloads scenarios

Attention on Blackwell

Level	Concept	Variations
Attention Algorithm-specific	SeqlenKind	Fixlen • Varlen • VarlenPaged
	MaskKind	Causal • NoMask • Sliding Window • Strided
	GQAPacking	Packed • Not Packed
	Density	Dense • Vector Sparse • Block Sparse • Hierarchical
Scheduling-specific	Tile Scheduler (Grid-level scheduling)	Static Persistent • Dynamic Persistent
	Kernel Schedule (Warp-specialization)	1xSoftmax • 2xSoftmax • Exp-Transform • 2xQ
Low-level hardware-specific	Store output	TMA-STORE • STG
	Math	UMMA.2CTA • UMMA • MMA.SYNC • Mixed
	Load input	TMA-LOAD • ASYNC-COPY • Mixed
	Dtype	BF16 • F8 • F4 • Mixed
	Tensor Memory (TMEM) partitioning	128x512x32b TMEM with different partitioning

Attention on Blackwell

Level	Concept	Variations
Attention Algorithm-specific	SeqlenKind	Fixlen • Varlen • VarlenPaged
	MaskKind	Causal • NoMask • Sliding Window • Strided
	GQAPacking	Packed • Not Packed
	Density	Dense • Vector Sparse • Block Sparse • Hierarchical
Scheduling-specific	Tile Scheduler (Grid-level scheduling)	Static Persistent • Dynamic Persistent
	Kernel Schedule (Warp-specialization)	1xSoftmax • 2xSoftmax • Exp-Transform • 2xQ
Low-level hardware-specific	Store output	TMA-STORE • STG
	Math	UMMA.2CTA • UMMA • MMA.SYNC • Mixed
	Load input	TMA-LOAD • ASYNC-COPY • Mixed
	Dtype	BF16 • F8 • F4 • Mixed
	Tensor Memory (TMEM) partitioning	128x512x32b TMEM with different partitioning

Attention on Blackwell

Level	Concept	Variations
Attention Algorithm-specific	SeqlenKind	
	MaskKind	
	GQAPacking	
	Density	
Scheduling-specific	Tile Scheduler (Grid-level scheduling)	
	Kernel Schedule (Warp-specialization)	
Low-level hardware-specific	Store output	
	Math	
	Load input	
	Dtype	
	Tensor Memory (TMEM) partitioning	

136 forward attention kernel variants for one dtype

16 forward attention foundational kernel variants in this talk

Attention on Blackwell

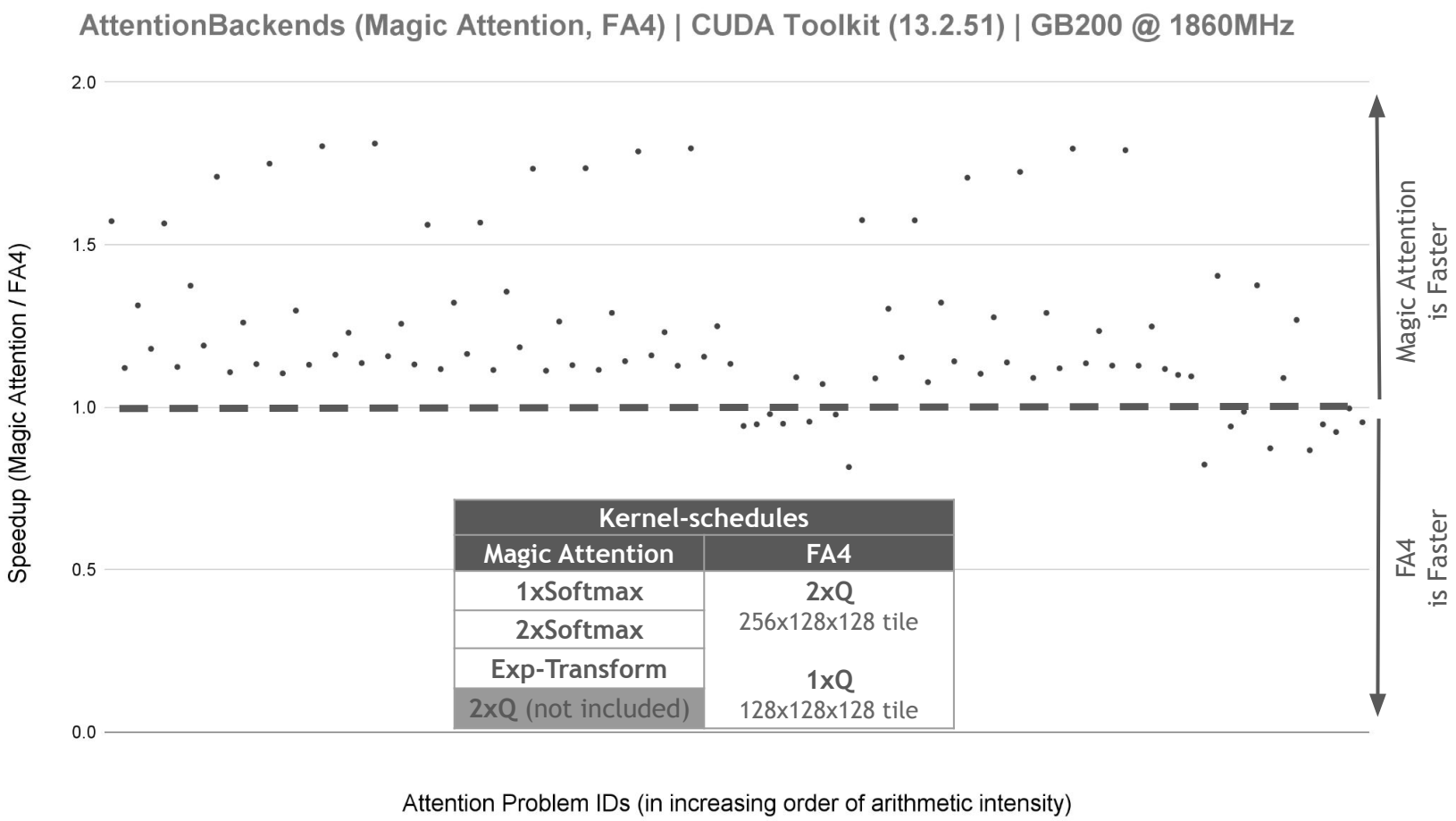
Level	Concept	Variations
Attention Algorithm-specific	SeqlenKind	
	MaskKind	
	GQAPacking	
	Density	
Scheduling-specific	Tile Scheduler (Grid-level scheduling)	
	Kernel Schedule (Warp-specialization)	
Low-level hardware-specific	Store output	
	Math	
	Load input	
	Dtype	
	Tensor Memory (TMEM) partitioning	

CUDA Graph
out of the box
for all variants

Static Persistent
SM carve-out support

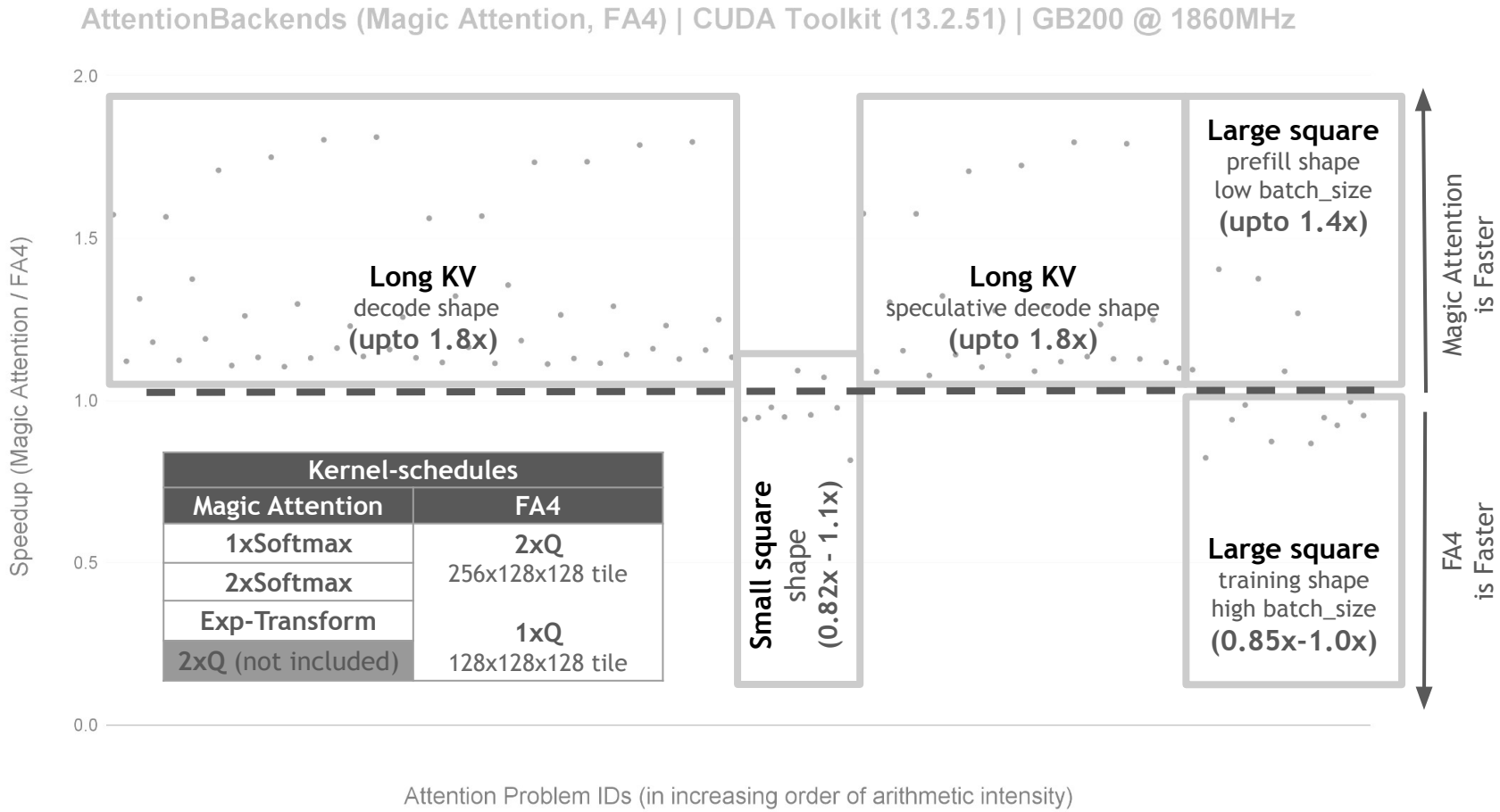
Dynamic Persistent
Cluster Launch Control (CLC)
Efficient draining of oversized grid

Performance on GB200



Magic

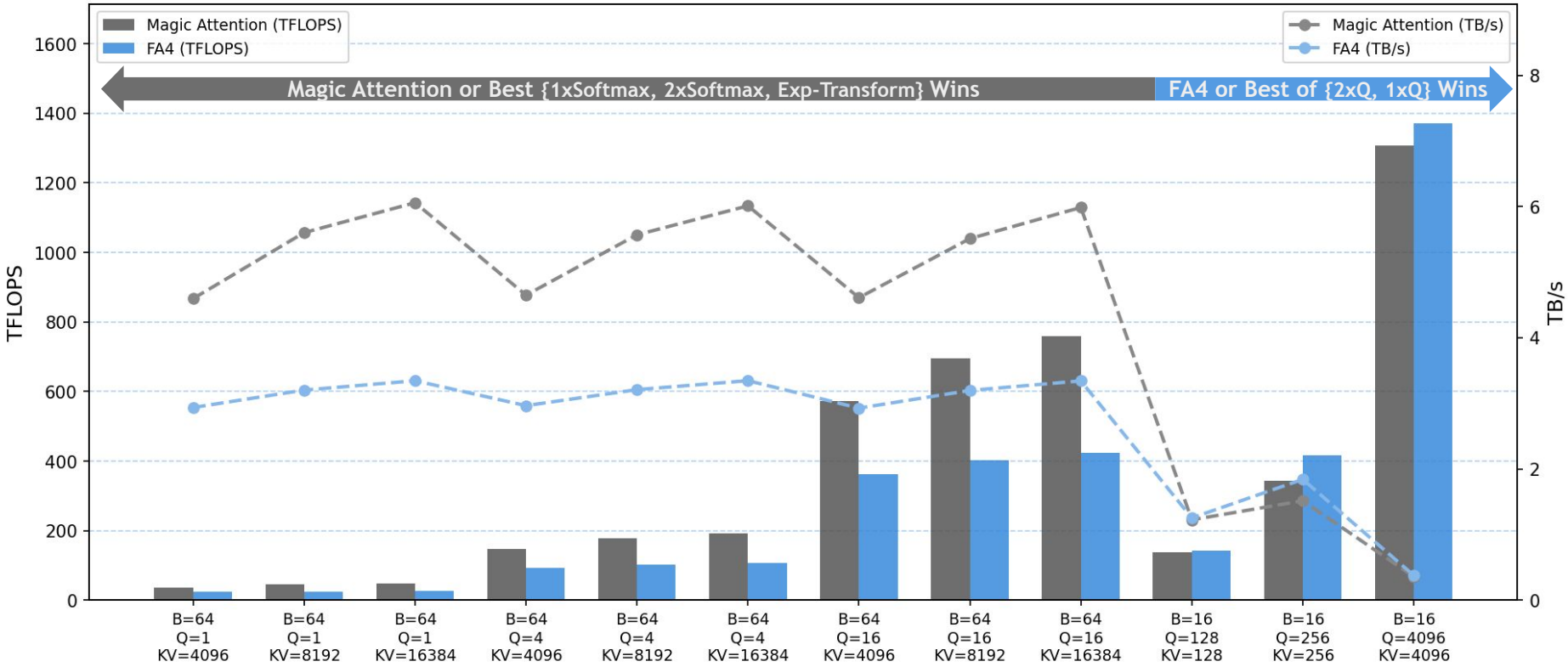
Performance on GB200



Magic

Performance on GB200

AttentionBackends=(Magic Attention, FA4) | CTK=13.2.51 | CLK=1860MHz | warmup-time=25ms | profiling-time=1000ms | cudagraph enabled



Magic

Agenda

- Attention Algorithm
- Tiling Attention on a GPU
- Convolution-inspired Runtime GQA Packing
- Attention on NVIDIA Blackwell Architecture
- Feeding the Datapath on NVIDIA Blackwell GB200

Attention Operation

$$\mathbf{QK} = \mathbf{Q} \mathbin{@} \mathbf{K}^{T}$$

$$\mathbf{E} = \mathrm{Softmax}(\mathbf{QK}) \,/\, \mathrm{sqrt}(H)$$

$$\mathbf{A} = \mathrm{Transform}(\mathbf{E})$$

$$\mathbf{AV} \mathrel{+}= \mathbf{A} \mathbin{@} \mathbf{V}$$

Attention Operation

$$QK = Q \mathbin{@} K^T$$

$$E = \mathrm{Softmax}(QK) / \mathrm{sqrt}(H)$$

$$\boxed{A = \mathrm{Transform}(E)}$$

$$AV \mathrel{+}= A \mathbin{@} V$$

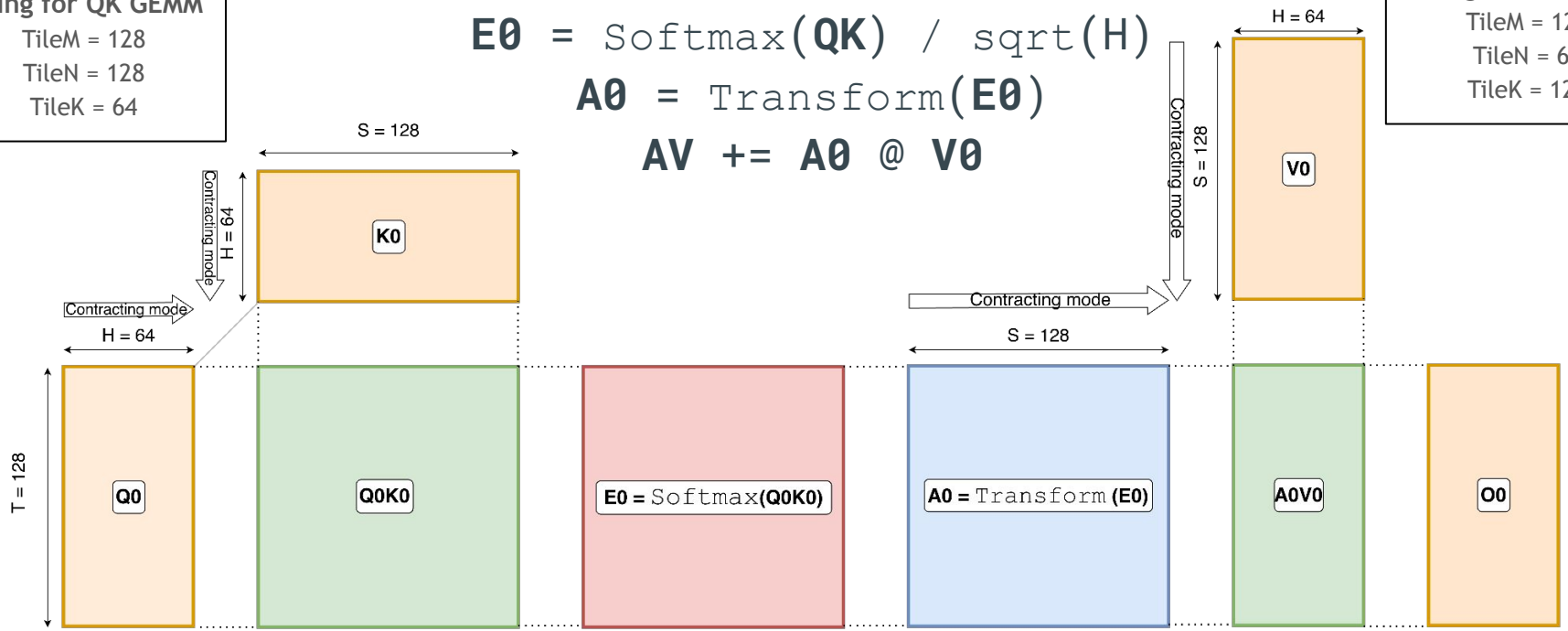
- `Transform` does not alter the core attention algorithm
- **Cleaner abstractions** enables **crazier warp-specializations**

Attention Tiling - Journey of a Tile

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

$$QK = Q0 \ @ \ K0^T$$
$$E0 = \mathrm{Softmax}(QK) \ / \ \mathrm{sqrt}(H)$$
$$A0 = \mathrm{Transform}(E0)$$
$$AV \mathrel{+}= A0 \ @ \ V0$$

Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128



T : Query tokens (128)
S : Key-value tokens (128)
H : Head dimension (64)

N : Number of Q heads (1)
K : Number of KV heads (1)
G : Number of Q heads per KV heads (N // K = 1)

Notations from the JAX ebook [How To Scale Your Model](#)'s Transform Accounting section

Magic

Attention Tiling - Journey of a Tile

$$QK = Q0 \ @ \ K0^T$$

$$QK = Mask(QK)$$

$$E0 = Softmax(QK) \ / \ sqrt(H)$$

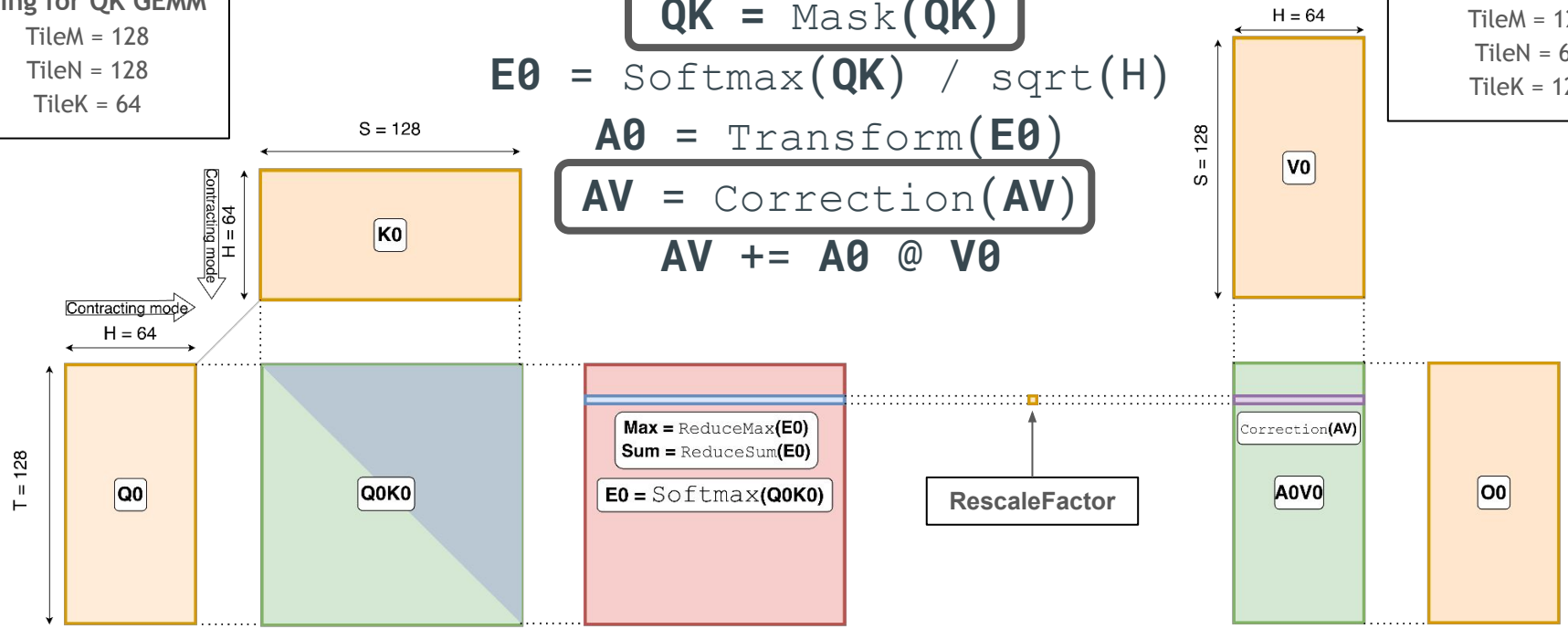
$$A0 = Transform(E0)$$

$$AV = Correction(AV)$$

$$AV \mathrel{+}= A0 \ @ \ V0$$

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128



T : Query tokens (128)
S : Key-value tokens (128)
H : Head dimension (64)

N : Number of Q heads (1)
K : Number of KV heads (1)
G : Number of Q heads per KV heads (N // K = 1)

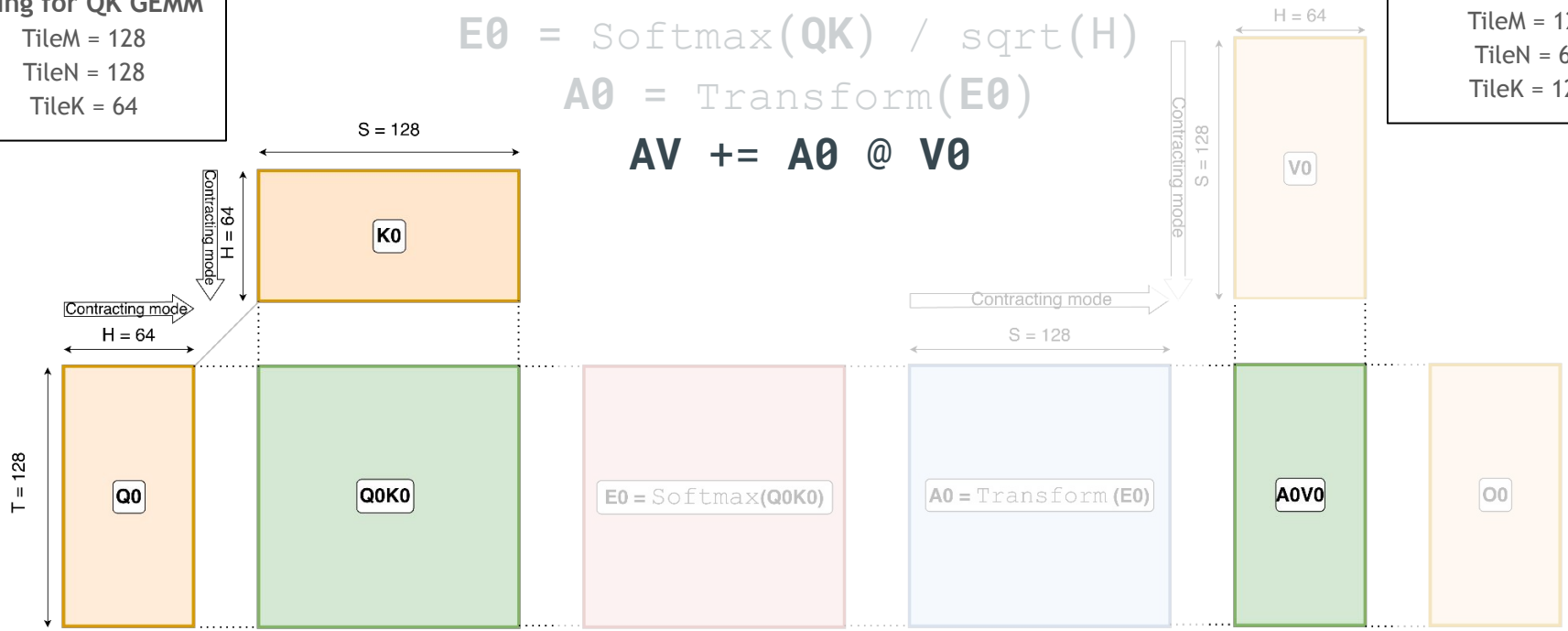
FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Tri Dao et. al.

Attention Tiling - Journey of a Tile

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

$$QK = Q0 \ @ \ K0^T$$
$$E0 = Softmax(QK) \ / \ sqrt(H)$$
$$A0 = Transform(E0)$$
$$AV \ += \ A0 \ @ \ V0$$

Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128



T : Query tokens (128)
S : Key-value tokens (128)
H : Head dimension (64)

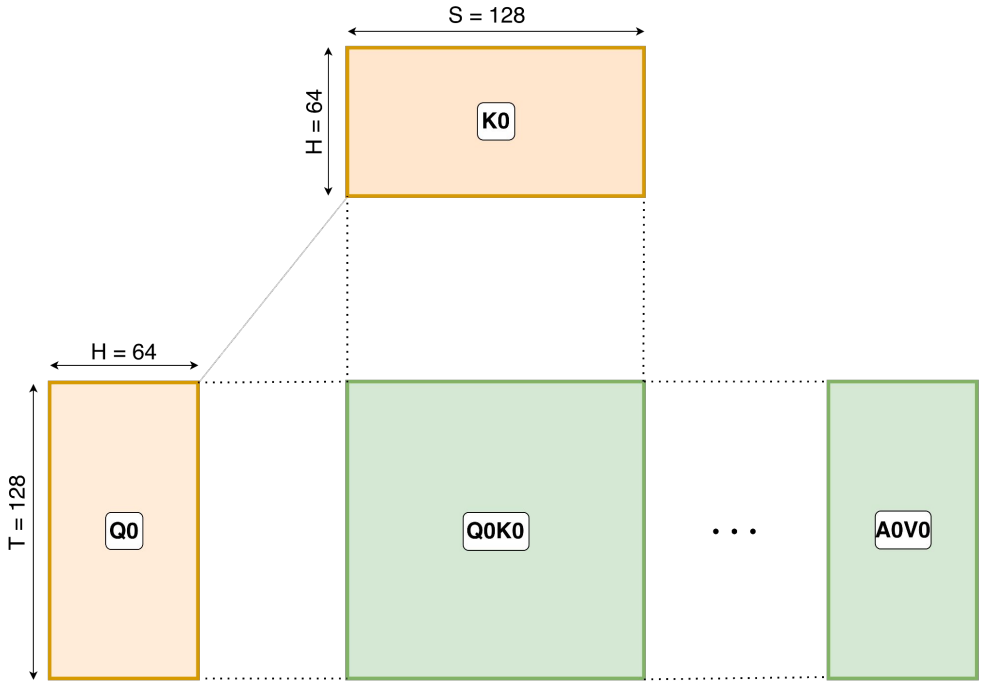
N : Number of Q heads (1)
K : Number of KV heads (1)
G : Number of Q heads per KV heads (N // K = 1)

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Tri Dao et. al.

Attention Tiling - 1 Q Tile, 1 KV Tile

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128



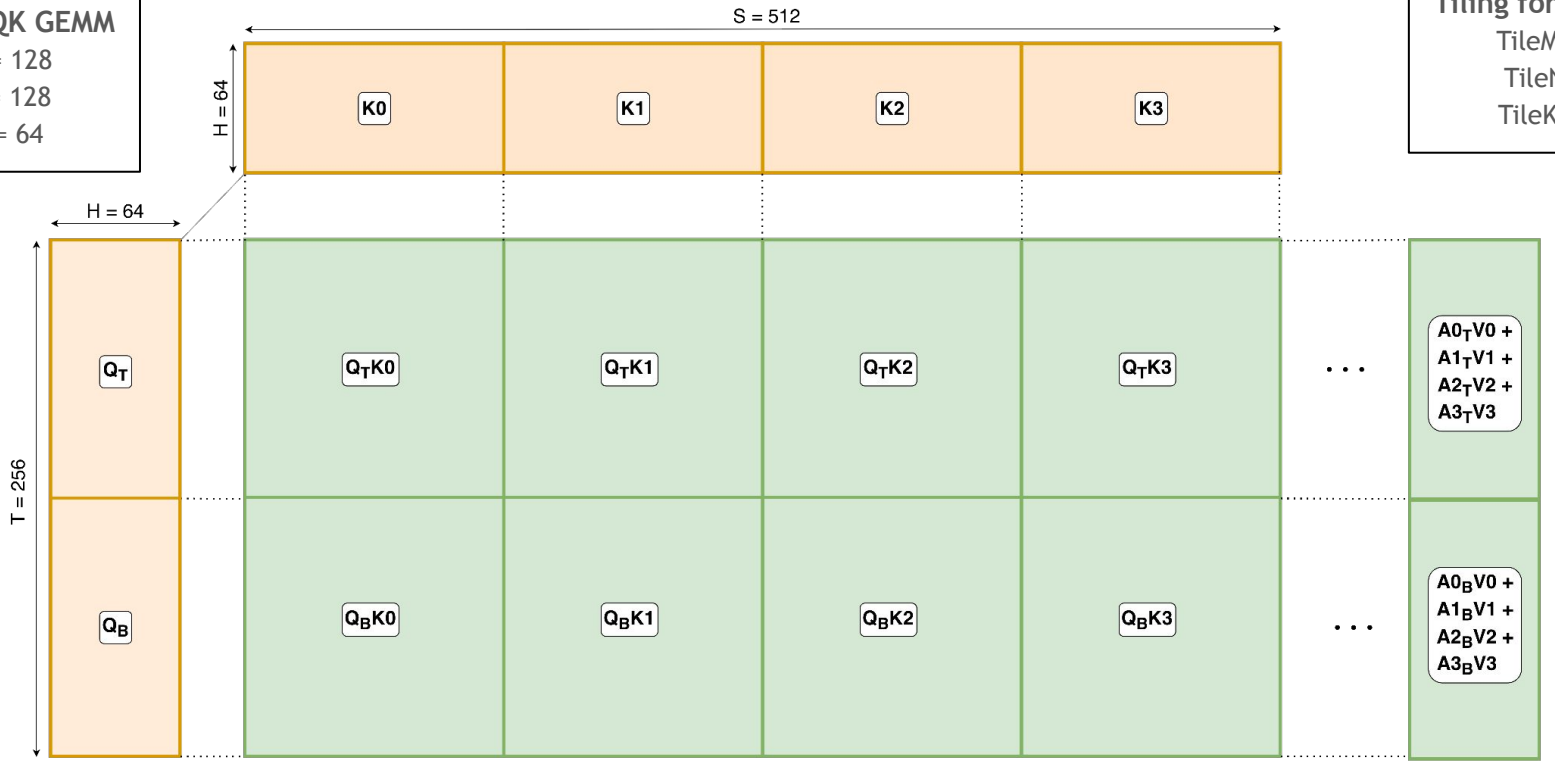
T : Query tokens (128)
S : Key-value tokens (128)
H : Head dimension (64)

N : Number of Q heads (1)
K : Number of KV heads (1)
G : Number of Q heads per KV heads (N // K = 1)

Attention Tiling - 2 Q Tile, 4 KV Tile

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128



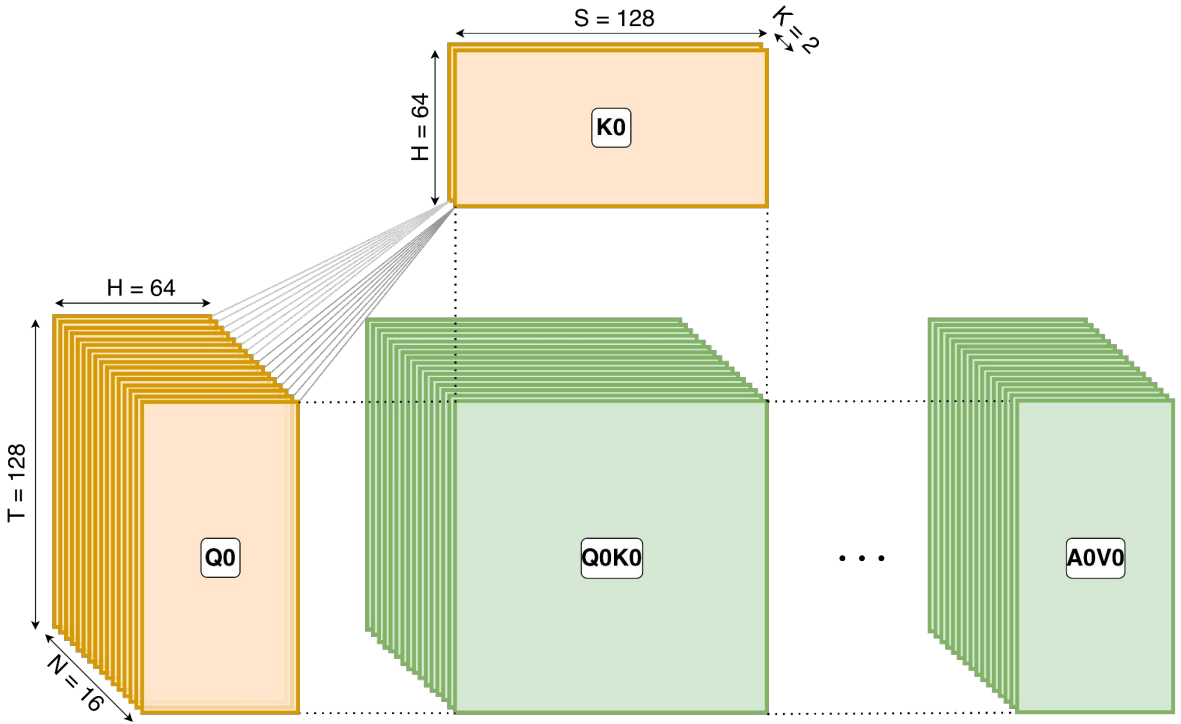
T : Query tokens (256)
S : Key-value tokens (512)
H : Head dimension (64)

N : Number of Q heads (1)
K : Number of KV heads (1)
G : Number of Q heads per KV heads (N // K = 1)

Attention Tiling - Multi-dimensional Tensors

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128



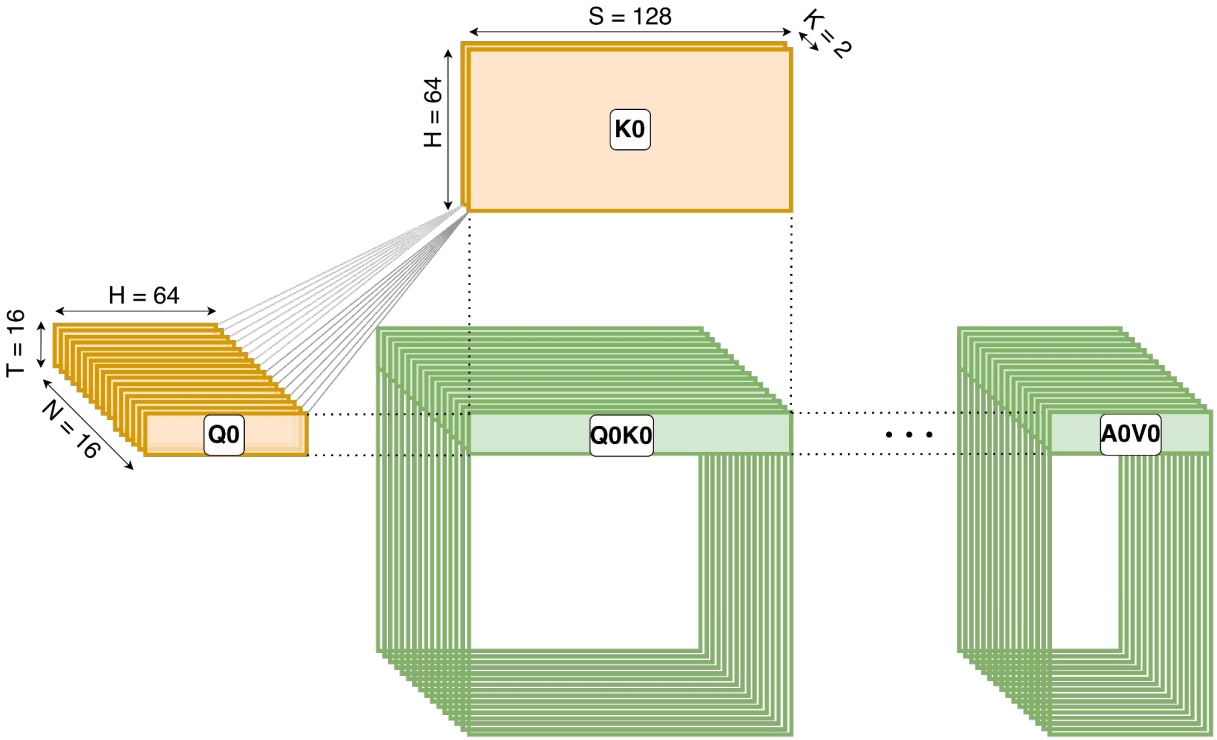
T : Query tokens (128)
S : Key-value tokens (128)
H : Head dimension (64)

N : Number of Q heads (16)
K : Number of KV heads (2)
G : Number of Q heads per KV-heads (N // K = 8)

Attention Tiling - Multi-dimensional Tensors

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

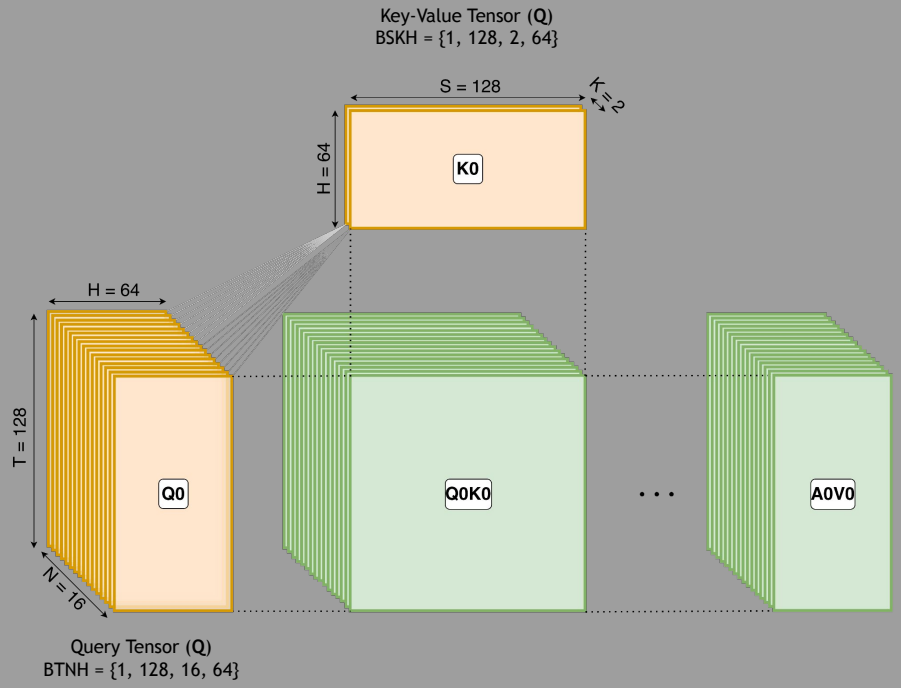
Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128



T : Query tokens (16)
S : Key-value tokens (128)
H : Head dimension (64)

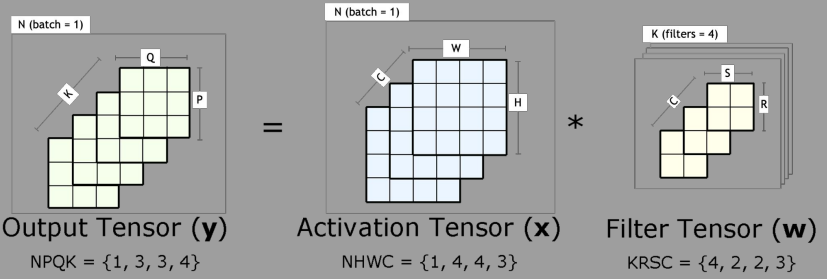
N : Number of Q heads (16)
K : Number of KV heads (2)
G : Number of Q heads per KV-heads (N // K = 8)

Digression from Attention to Convolution



B (Batch) = 1

Attention



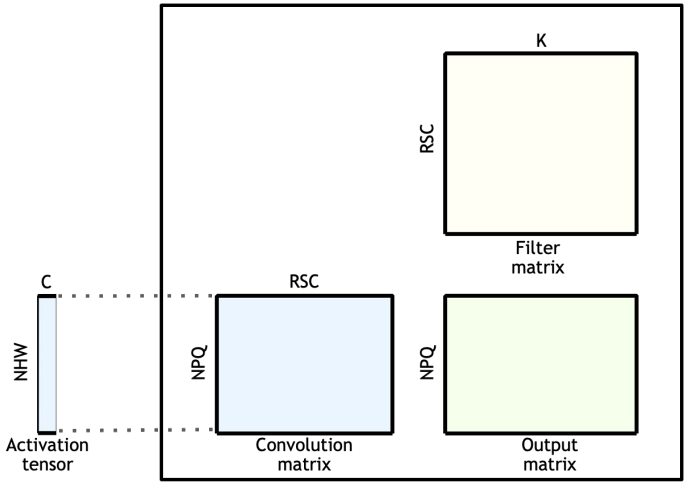
N (Batch) = 1

Convolution

4D Tensors to 2D Matrices at **Runtime**

The convolution operation on 4D tensors can be mapped as matrix-multiply operation on 2D matrices

4D Tensors	**2D Matrix**
y = CONV(**x**, **w**)	**C** = **A** @ **B**
x[N,H,W,C] : 4D activation tensor	**A**[NHW, RSC] : 2D convolution matrix
w[K,R,S,C] : 4D filter tensor	**B**[RSC, K] : 2D filter matrix
y[N,P,Q,K] : 4D output tensor	**C**[NPQ, K] : 2D output matrix

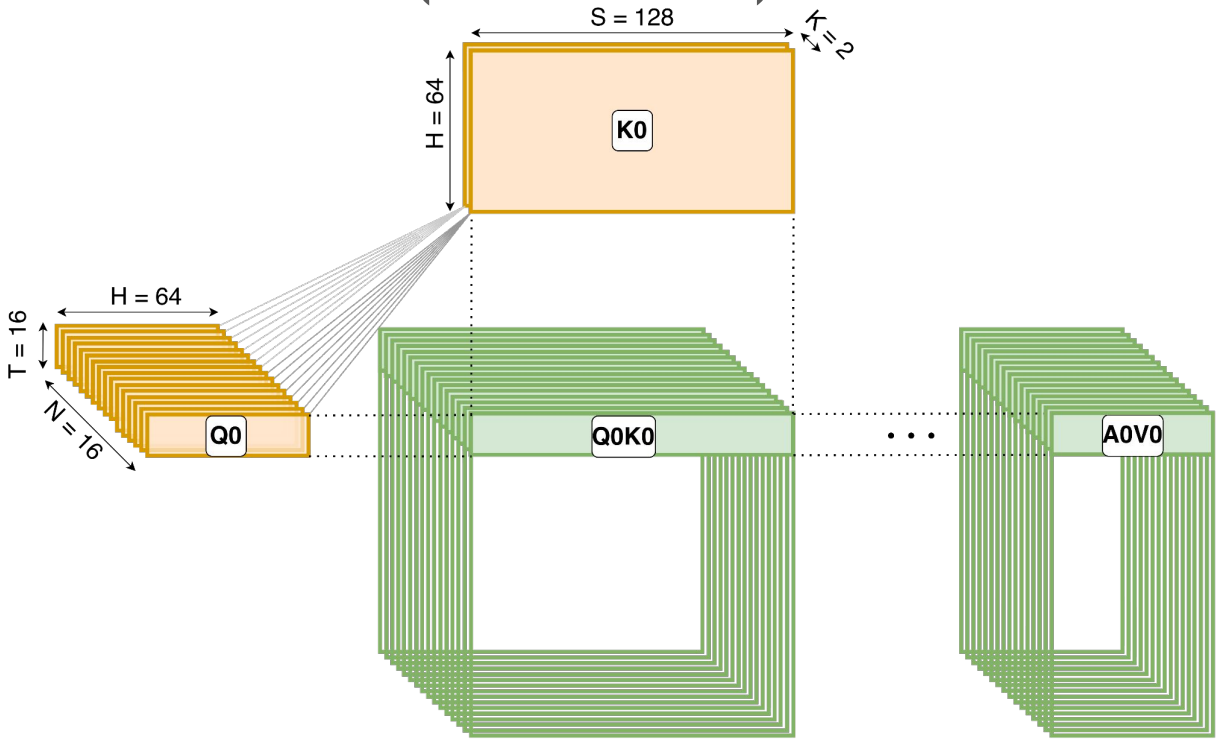


Source: Accelerating Convolution with Tensor Cores in CUTLASS, GTC 2021

Attention Tiling - Multi-dimensional Tensors (Not Packed)

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128



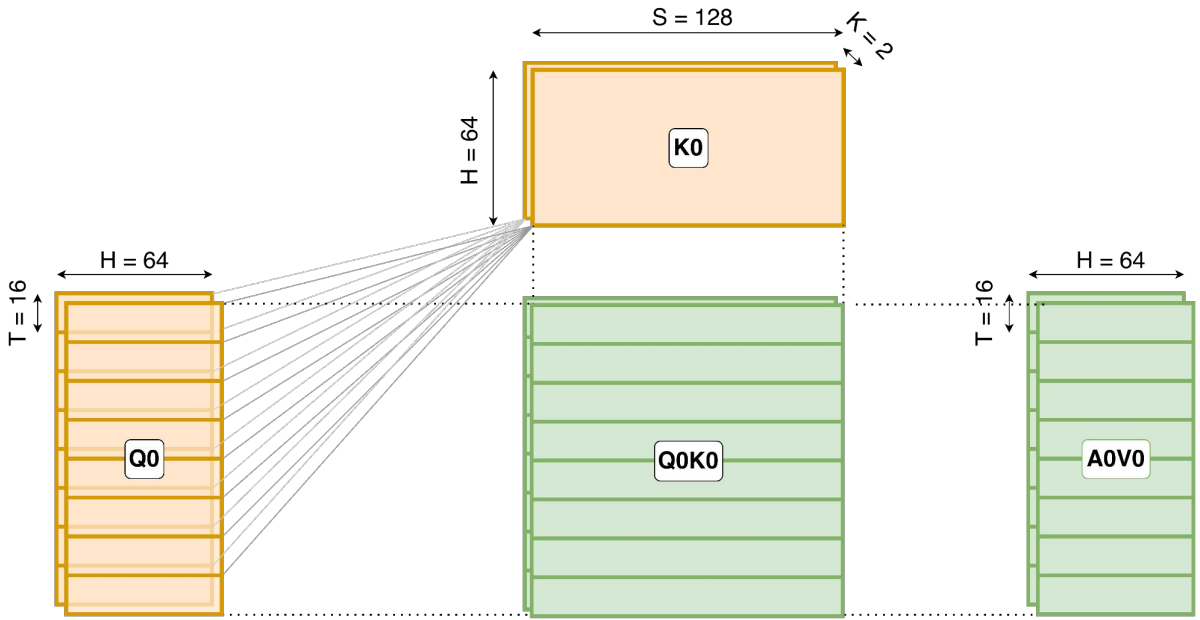
T : Query tokens (16)
S : Key-value tokens (128)
H : Head dimension (64)

N : Number of Q heads (16)
K : Number of KV heads (2)
G : Number of Q heads per KV-heads (N // K = 8)

Attention Tiling - Multi-dimensional Tensors (Packed)

Tiling for QK GEMM
TileM = 128
TileN = 128
TileK = 64

Tiling for QK GEMM
TileM = 128
TileN = 64
TileK = 128

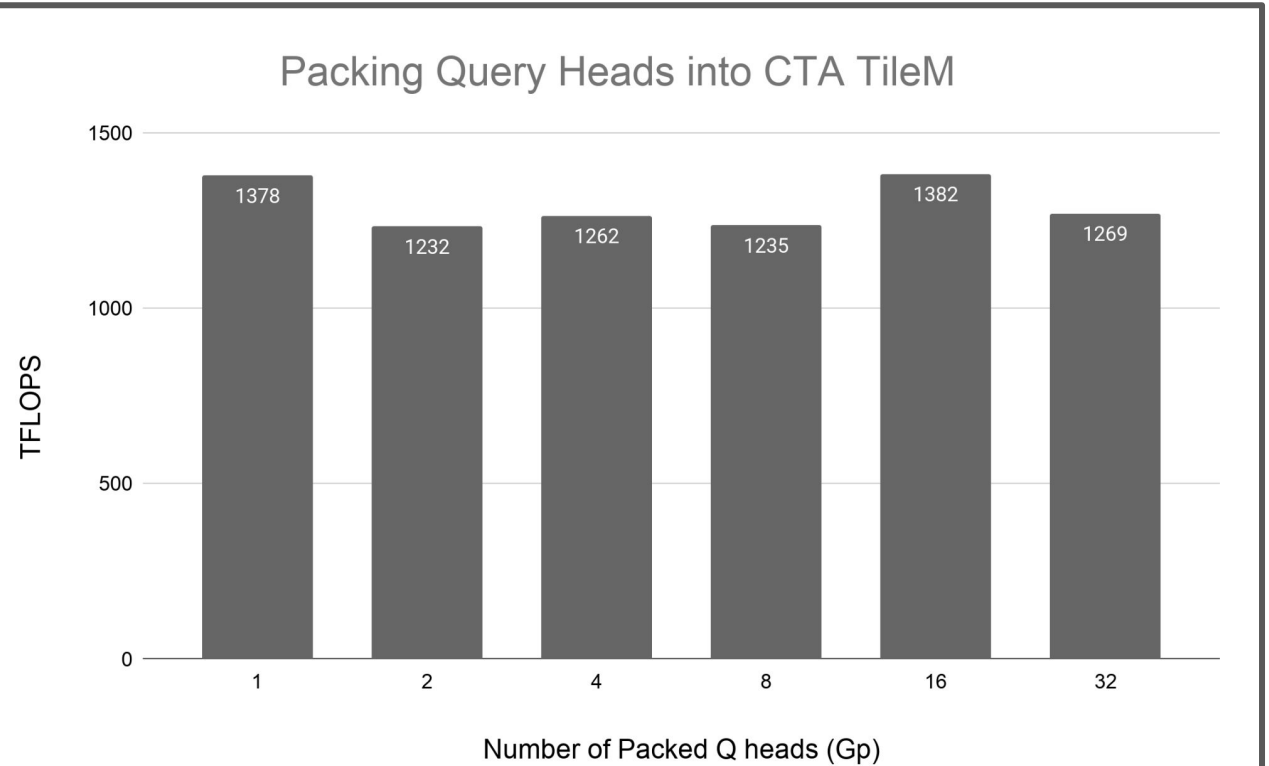


T : Query tokens (16)
S : Key-value tokens (128)
H : Head dimension (64)

N : Number of Q heads (16)
K : Number of KV heads (2)
G : Number of Q heads per KV-heads (N // K = 8)

Gp : Number of Packed Q heads (8)
Gu : Number of Un-packed Q heads (1)
G = Gp * Gu (8)

Performance with Runtime GQAPacking



Takeaways

1. Peak performance isn't always at max packing.
2. **GQAPacking *improves* CTA utilization** but ***degrades*** **grid parallelism.**
3. **Split-KV** with GQAPacking ***restores*** **grid parallelism.**
4. **Magic-Attention** introduces a convolution-inspired **runtime knob to tune GQAPacking for peak performance.**

T : Query tokens (48)
S : Key-value tokens (8192)
H : Head dimension (128)

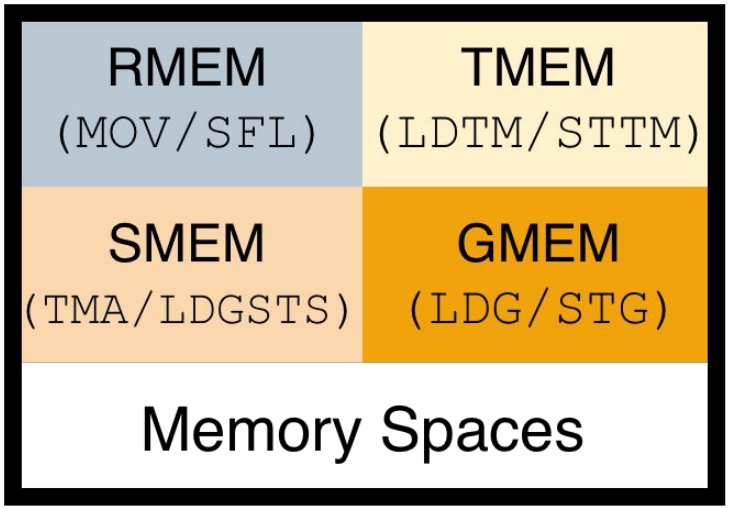
N : Number of Q heads (32)
K : Number of KV heads (1)
G : Number of Q heads per KV-heads (N // K = 32)

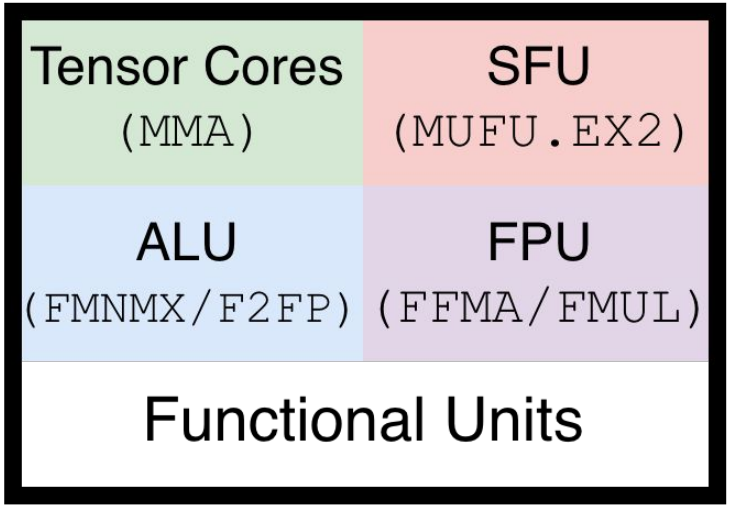
Gp : Number of Packed Q heads (1,2,4,8,16,32)
Gu : Number of Unpacked Q heads (32,16,8,4,2,1)
G = Gp * Gu

Agenda

- Attention Algorithm
- Tiling Attention on a GPU
- Convolution-inspired Runtime GQA Packing
- Attention on NVIDIA Blackwell Architecture
- Feeding the Datapath on NVIDIA Blackwell GB200

NVIDIA Blackwell GPU



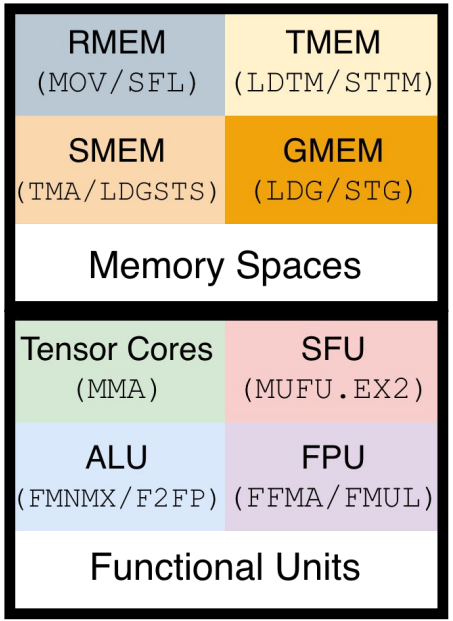


`LDTM`	:	Load TMEM ⟶ RMEM
`STTM`	:	Store RMEM ⟶ TMEM
`TMA`	:	Load/Store GMEM ⟷ SMEM `cp.async.bulk`
`LDGSTS`	:	Load GMEM ⟶ SMEM with `cp.async`
`LDG`	:	Load GMEM ⟶ RMEM
`STG`	:	Store RMEM ⟶ GMEM

`MMA`	:	Tensor Core Matrix Multiply Accumulate
`MUFU.EX2`	:	Exponentiation operation to the base 2
`FMNMX`	:	Floating-point min-max
`F2FP`	:	Floating-point conversion
`FADD`	:	Floating point add
`FMUL`	:	Floating point multiply
`FFMA`	:	Fused floating point multiply add

Attention Algorithm on the NVIDIA Blackwell SM

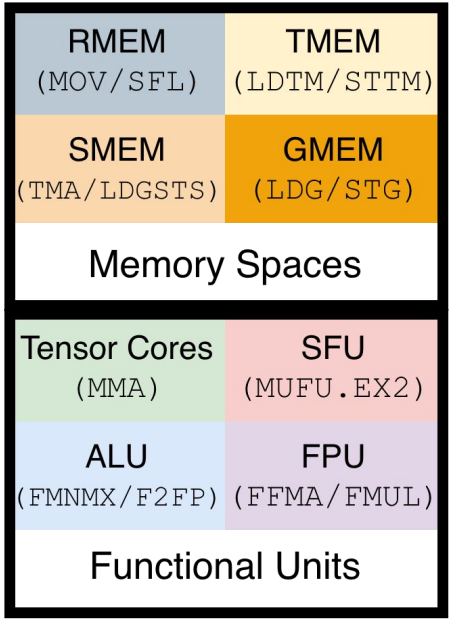
$$QK_{TMEM} = Q_{SMEM} @ K_{SMEM}^{T}$$



Attention Algorithm on the NVIDIA Blackwell SM

$$QK_{TMEM} = Q_{SMEM} \; @ \; K_{SMEM}^{T}$$

$$QK_{RMEM} = \mathtt{LDTM}(QK_{TMEM})$$

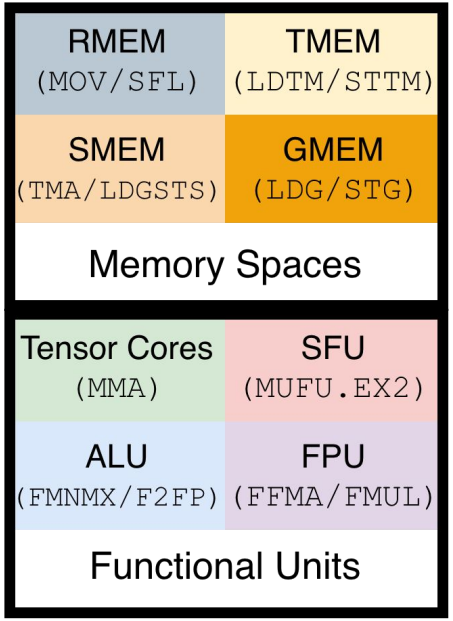


Attention Algorithm on the NVIDIA Blackwell SM

$$QK_{TMEM} = Q_{SMEM} \; @ \; K_{SMEM}^{T}$$

$$QK_{RMEM} = \mathtt{LDTM}(QK_{TMEM})$$

$$E_{RMEM} = \mathtt{Softmax}(QK_{RMEM})/\mathtt{sqrt}(H)$$



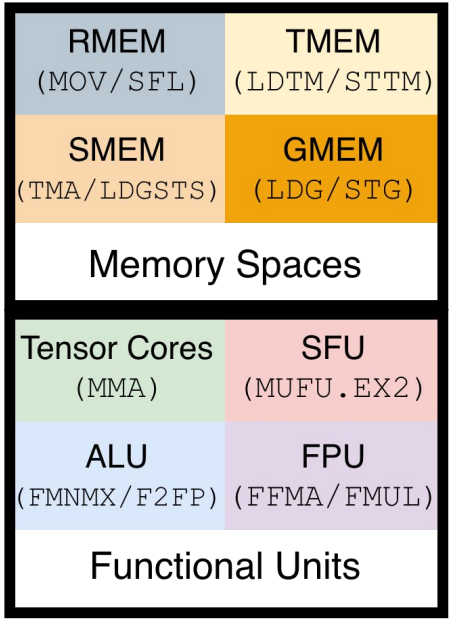
Attention Algorithm on the NVIDIA Blackwell SM

$$QK_{TMEM} = Q_{SMEM} \; @ \; K_{SMEM}^{T}$$

$$QK_{RMEM} = \mathrm{LDTM}(QK_{TMEM})$$

$$E_{RMEM} = \mathrm{Softmax}(QK_{RMEM})/\mathrm{sqrt}(H)$$

$$A_{RMEM} = \mathrm{Transform}(E_{RMEM})$$



Attention Algorithm on the NVIDIA Blackwell SM

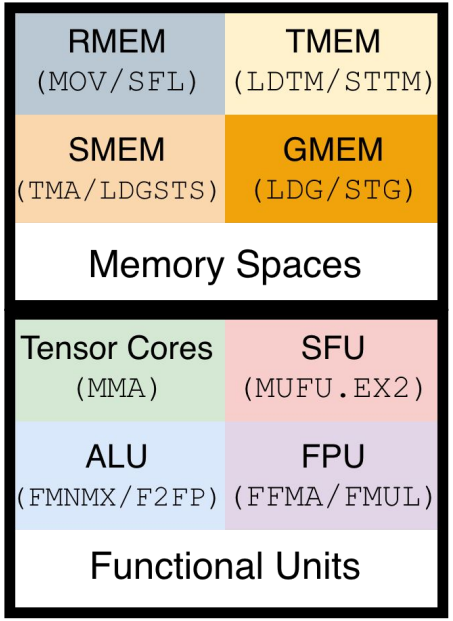
$$QK_{TMEM} = Q_{SMEM} \; @ \; K_{SMEM}^{T}$$

$$QK_{RMEM} = \texttt{LDTM}(QK_{TMEM})$$

$$E_{RMEM} = \texttt{Softmax}(QK_{RMEM})/\texttt{sqrt}(H)$$

$$A_{RMEM} = \texttt{Transform}(E_{RMEM})$$

$$A_{TMEM} = \texttt{STTM}(A_{RMEM})$$



Attention Algorithm on the NVIDIA Blackwell SM

$$QK_{TMEM} = Q_{SMEM} \; @ \; K_{SMEM}^{T}$$

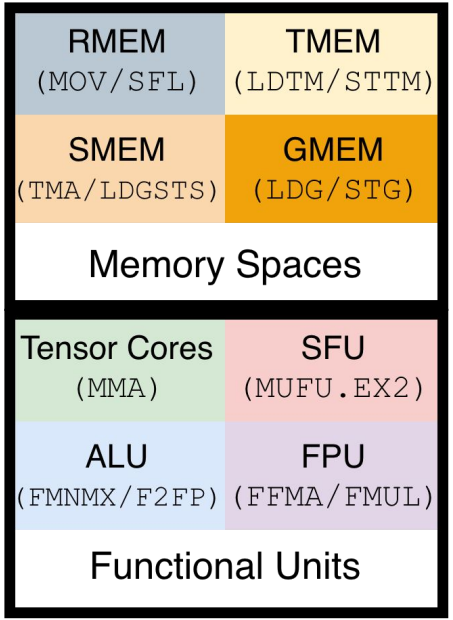
$$QK_{RMEM} = \texttt{LDTM}(QK_{TMEM})$$

$$E_{RMEM} = \texttt{Softmax}(QK_{RMEM})/\texttt{sqrt}(H)$$

$$A_{RMEM} = \texttt{Transform}(E_{RMEM})$$

$$A_{TMEM} = \texttt{STTM}(A_{RMEM})$$

$$AV_{TMEM} \mathrel{+}= A_{TMEM} \; @ \; V_{SMEM}$$



Attention Algorithm on the NVIDIA Blackwell SM

$$QK_{TMEM} = Q_{SMEM} @ K_{SMEM}^{T}$$

$$QK_{RMEM} = LDTM(QK_{TMEM})$$

$$E_{RMEM} = Softmax(QK_{RMEM})/sqrt(H)$$

$$A_{RMEM} = Transform(E_{RMEM})$$

$$A_{TMEM} = STTM(A_{RMEM})$$

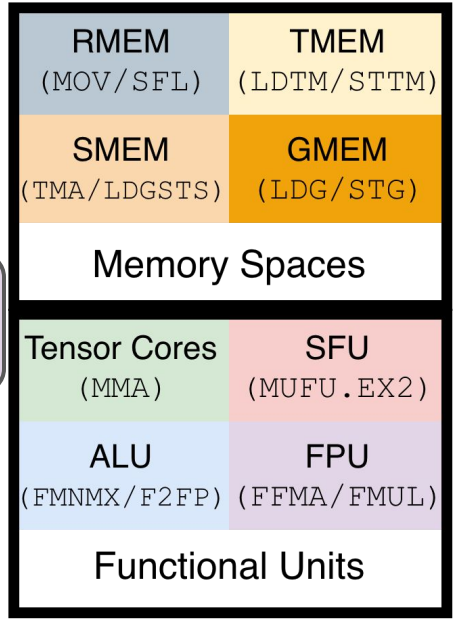
Correction

$$AV_{RMEM} = LDTM(AV_{TMEM})$$

$$AV_{RMEM} = AV_{RMEM} * RescaleFactor_{RMEM}$$

$$AV_{TMEM} = STTM(AV_{RMEM})$$

$$AV_{TMEM} \mathrel{+}= A_{TMEM} @ V_{SMEM}$$



Magic

Attention Algorithm on the NVIDIA Blackwell SM

$$QK_{TMEM} = Q_{SMEM} \; @ \; K_{SMEM}^{T}$$

$$QK_{RMEM} = \texttt{LDTM}(QK_{TMEM})$$

$$E_{RMEM} = \texttt{Softmax}(QK_{RMEM})/\texttt{sqrt}(H)$$

$$A_{RMEM} = \texttt{Transform}(E_{RMEM})$$

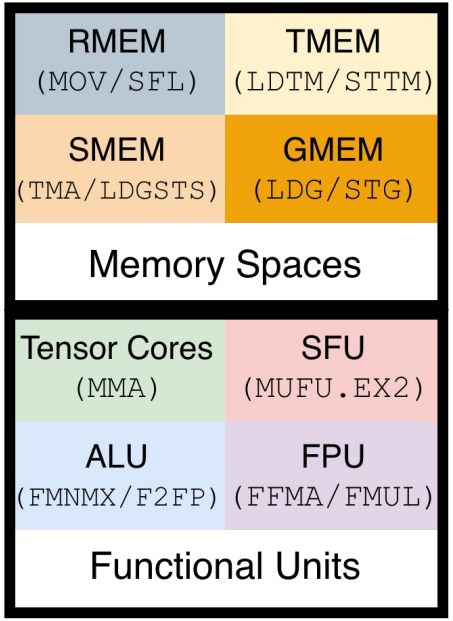
$$A_{TMEM} = \texttt{STTM}(A_{RMEM})$$

$$AV_{RMEM} = \texttt{LDTM}(AV_{TMEM})$$

$$AV_{RMEM} = \texttt{Correction}(AV_{RMEM})$$

$$AV_{TMEM} = \texttt{STTM}(AV_{RMEM})$$

$$AV_{TMEM} \mathrel{+}= A_{TMEM} \; @ \; V_{SMEM}$$



Agenda

- Attention Algorithm
- Tiling Attention on a GPU
- Convolution-inspired Runtime GQA Packing
- Attention on NVIDIA Blackwell Architecture
- Feeding the Datapath on NVIDIA Blackwell GB200

Feeding the Data Path on NVIDIA Blackwell

- **Within**-warp optimizations
  - Vectorization (e.g. `FMNMX3`, `FADD2`, `FFMA2`, `F2FP.*.BF16.F32.PACK_AB`)
  - Instruction level parallelism (ILP)
  - Software pipelining
- **Across**-warp asynchrony (Warp-specialization)
  - 1xSoftmax
  - 2xSoftmax
  - Exp-Transform
  - 2xQ
- **Efficient** use of Tensor Memory (TMEM)
  - `256KB` of TMEM per SM
  - Many different partitioning possible with different trade-offs

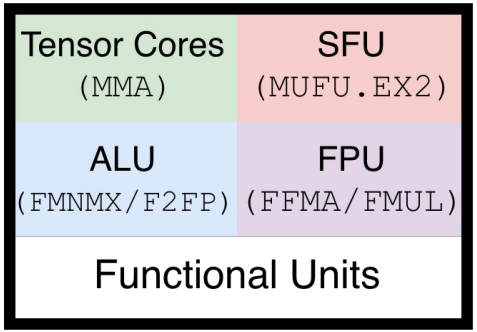
Feeding the Data Path on NVIDIA Blackwell

- **Within**-warp optimizations
  - Vectorization (e.g. `FMNMX3`, `FADD2`, `FFMA2`, `F2FP.*.BF16.F32.PACK_AB`)
  - Instruction level parallelism (ILP)
  - Software pipelining
- **Across**-warp asynchrony (Warp-specialization)
  - 1xSoftmax
  - 2xSoftmax
  - Exp-Transform
  - 2xQ
- **Efficient** use of Tensor Memory (TMEM)
  - `256KB` of TMEM per SM
  - Many different partitioning possible with different trade-offs

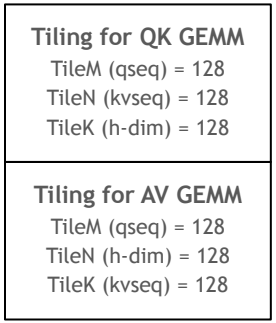
Roofline Analysis - NVIDIA GB200

Machine (GB200)	Peak rate (per-GPU)	Peak rate (per-SM, per-Cycle)
MMA **(BF16)**	2.5 PFLOPS	8192 FLOPS/SM/Cycle
MUFU.EX2	5 TEXPOPS	16 EXOPS/SM/Cycle
FFMA	80 TFLOPS	256 FLOPS/SM/Cycle
FMUL	40 TFLOPS	128 FLOPS/SM/Cycle

Attention Algorithm	Total OPS (per Tile)	Cycles (Total OPS/Peak rate)
MMA QK	2*128*128*128	**512**
MMA AV	2*128*128*128	**512**
MUFU.EX2	128*128	**1024**
FFMA ($x*scale - x_{max}$)	2*128*128	**128**
FMUL ($AV_{corr} = AV * rescale$)	128*128	**128**



NVIDIA Blackwell (**GB200**)



Attention Tiling Details

Within-Warp Optimizations - `Correction`

We can achieve nearly perfect Instruction-Level Parallelism (ILP) for `Correction` because of:

- Straight-forward dependency chains
- Double-buffering in RMEM, prefetching one stage from TMEM, and interleaving `LDTM`, `FMUL`, and `STTM`
- Exposes only a part of `LDTM` at the start and `STTM` at the end

Attention Algorithm	Total OPS (per Tile)	Cycles (Total OPS/Peak rate)
MMA QK	2*128*128*128	512
MMA AV	2*128*128*128	512
MUFU.EX2	128*128	1024
FFMA ($x*scale - x_{max}$)	2*128*128	128
FMUL (AV_{corr} = AV * rescale)	128*128	**128**


```
LDTM.x16 R20, tmem[UR5+0x100] ;
LDTM.x16 R4, tmem[UR5+0x110] ;
FMUL2 R20, R20.F32x2.HI_LO, R53.reuse.F32 ;
...
FMUL2 R6, R6.F32x2.HI_LO, R53.reuse.F32 ;
STTM.x16 tmem[UR5+0x100], R20 ;
FMUL2 R8, R8.F32x2.HI_LO, R53.reuse.F32 ;
LDTM.x16 R36, tmem[UR5+0x120] ;
FMUL2 R10, R10.F32x2.HI_LO, R53.F32 ;
...
FMUL2 R18, R18.F32x2.HI_LO, R53.F32 ;
STTM.x16 tmem[UR5+0x110], R4 ;
LDTM.x16 R20, tmem[UR5+0x130] ;
FMUL2 R36, R36.F32x2.HI_LO, R53.reuse.F32 ;
...
FMUL2 R22, R22.F32x2.HI_LO, R53.reuse.F32 ;
STTM.x16 tmem[UR5+0x120], R36 ;
FMUL2 R24, R24.F32x2.HI_LO, R53.reuse.F32 ;
LDTM.x16 R4, tmem[UR5+0x140] ;
FMUL2 R26, R26.F32x2.HI_LO, R53.F32 ;
...
FMUL2 R34, R34.F32x2.HI_LO, R53.F32 ;
STTM.x16 tmem[UR5+0x130], R20 ;
LDTM.x16 R36, tmem[UR5+0x150] ;
FMUL2 R4, R4.F32x2.HI_LO, R53.reuse.F32 ;
...
FMUL2 R38, R38.F32x2.HI_LO, R53.reuse.F32 ;
STTM.x16 tmem[UR5+0x140], R4 ;
FMUL2 R40, R40.F32x2.HI_LO, R53.reuse.F32 ;
LDTM.x16 R20, tmem[UR5+0x160] ;
FMUL2 R42, R42.F32x2.HI_LO, R53.F32 ;
...
FMUL2 R50, R50.F32x2.HI_LO, R53.F32 ;
STTM.x16 tmem[UR5+0x150], R36 ;
LDTM.x16 R4, tmem[UR5+0x170] ;
FMUL2 R20, R20.F32x2.HI_LO, R53.reuse.F32 ;
...
FMUL2 R6, R6.F32x2.HI_LO, R53.reuse.F32 ;
STTM.x16 tmem[UR5+0x160], R20 ;
FMUL2 R8, R8.F32x2.HI_LO, R53.F32 ;
...
FMUL2 R18, R18.F32x2.HI_LO, R53.F32 ;
STTM.x16 tmem[UR5+0x170], R4 ;
```

Our GPUs 💚 beautiful SASS

Within-Warp Optimizations - Softmax

Achieving a good ILP for the Softmax is difficult because:

- Dependencies on LDTM and FMNMX, before the first MUFU
- Too many instruction kinds in the mix
- Too many MUFU and too few ALU and FPU OPS

Attention Algorithm	Total OPS (per Tile)	Cycles (Total OPS/Peak rate)
MMA QK	2*128*128*128	512
MMA AV	2*128*128*128	512
MUFU.EX2	128*128	**1024**
FFMA (x*scale - x_{max})	2*128*128	**128**
FMUL (AV_{corr} = AV * rescale)	128*128	128

MUFU block theoretical

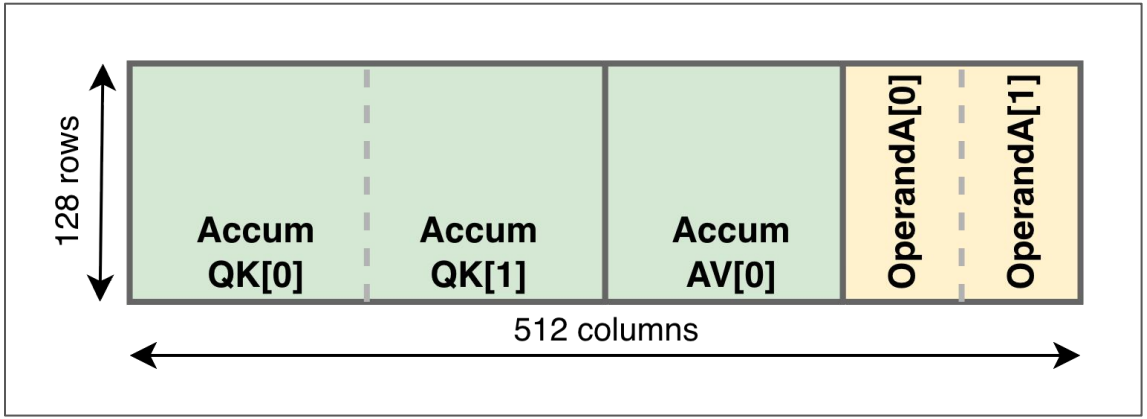
MUFU block in Attention Softmax

MUFU block in Attention Softmax with partial emulation

Feeding the Data Path on NVIDIA Blackwell

- **Within**-warp optimizations
  - Vectorization (e.g. `FMNMX3`, `FADD2`, `FFMA2`, `F2FP.*.BF16.F32.PACK_AB`)
  - Instruction level parallelism (ILP)
  - Software pipelining
- **Across**-warp asynchrony (Warp-specialization)
  - 1xSoftmax
  - 2xSoftmax
  - Exp-Transform
  - 2xQ
- **Efficient** use of Tensor Memory (TMEM)
  - `256KB` of TMEM per SM
  - Many different partitioning possible with different trade-offs

Tensor Memory on Blackwell

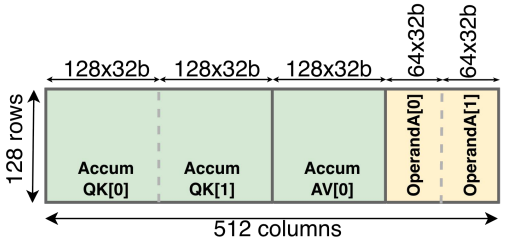


- **Easier pipelining:** promotes creating “crazier” warp-specialized schedules
- **Faster data transfer:** allows warpgroups to communicate data
- **Lower register pressure:** Offloads the accumulators from RMEM to TMEM

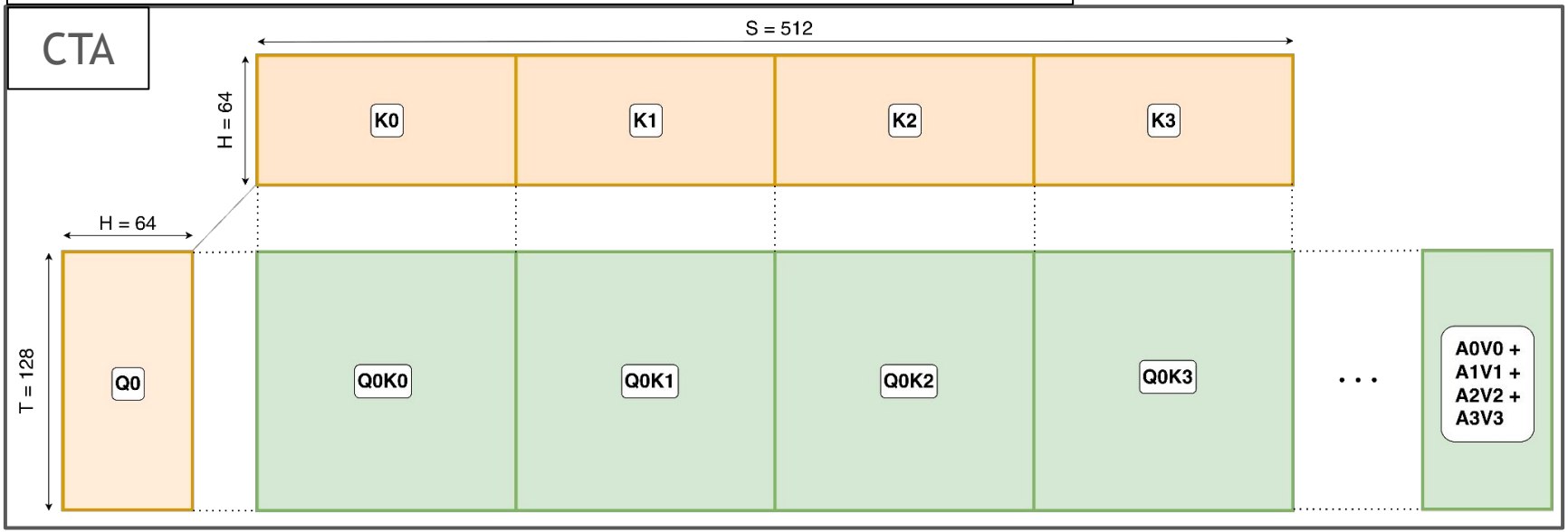
1xSoftmax, 2xSoftmax, Exp-Transform

1SM loads 1 Q tile, loads 4 KV tiles

- 1xQ tile per SM
- Lower SMEM usage for Q tiles, Higher SMEM usage for KV
  - Latency hiding by PreFetching more KV tiles
- TMEM partitioning
  - Two slots for AccumQK,
  - Two slots for OperandA
  - One slot for AccumAV

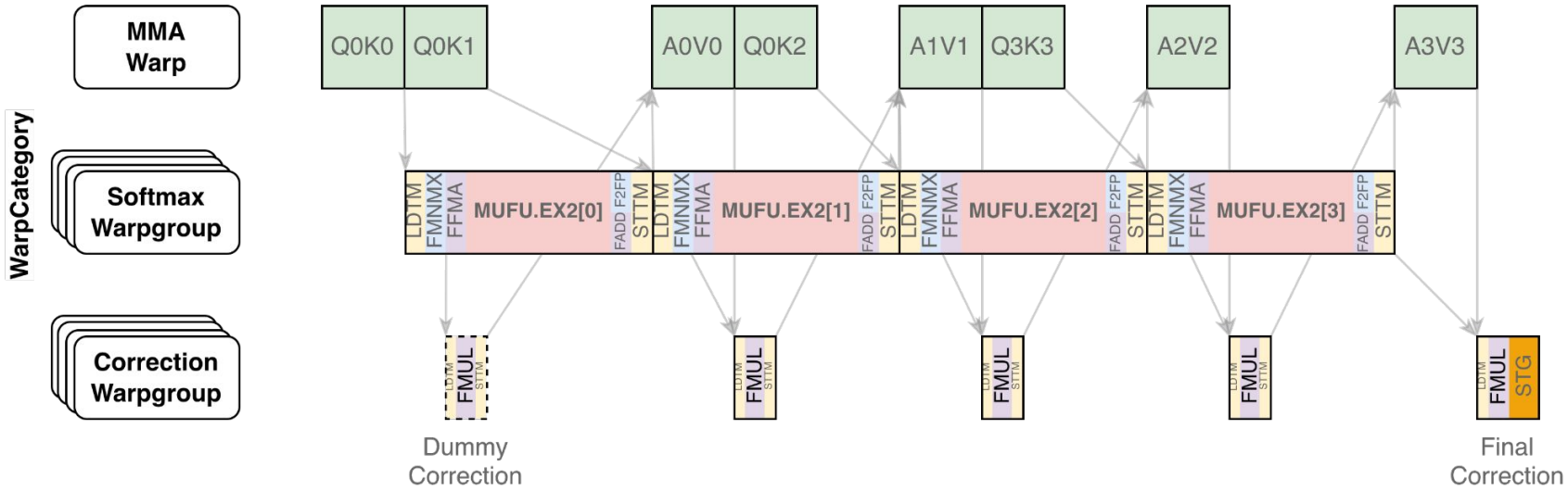


TMEM Partitioning



1xSoftmax

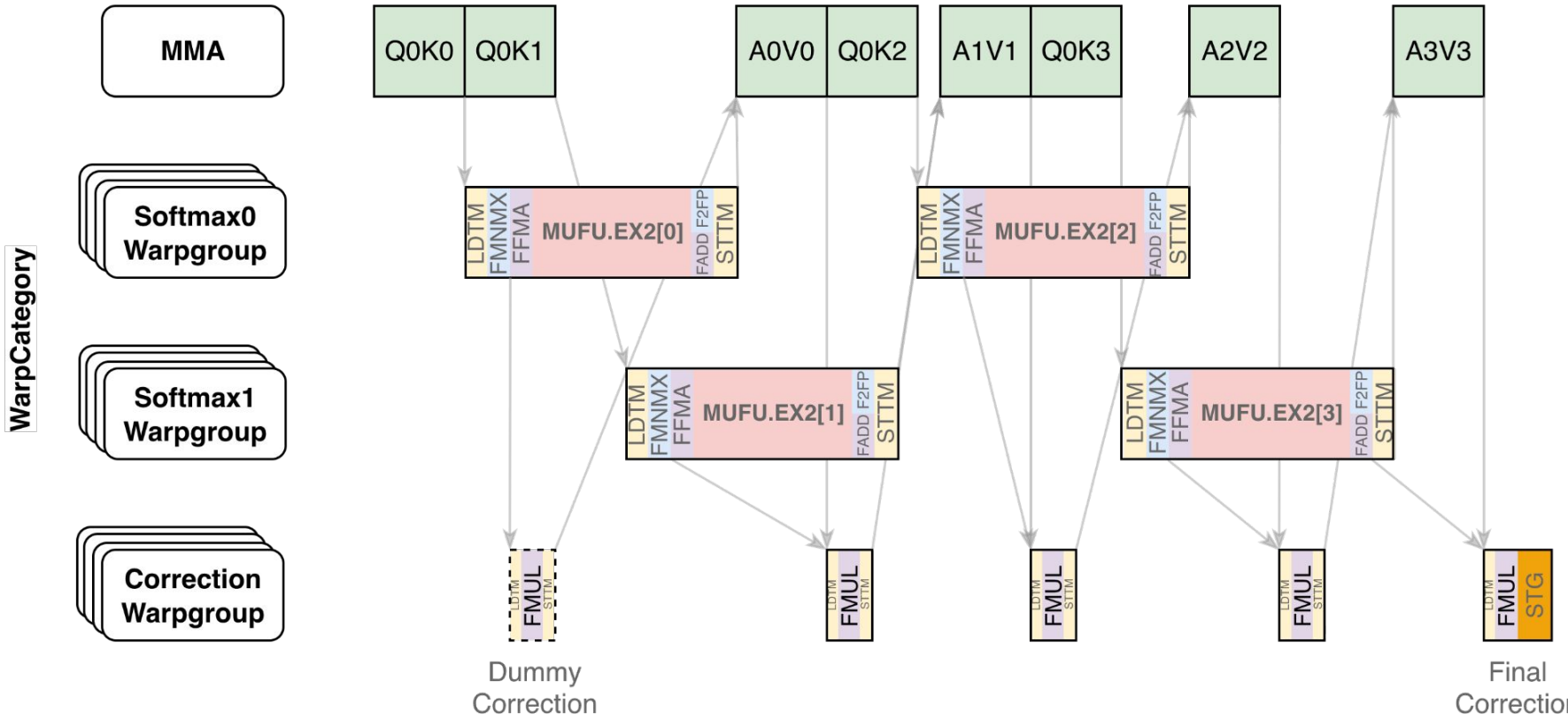
1SM loads 1 Q tile, loads 4 KV tiles, computes 4 softmax tiles with 1 softmax warpgroup



- Simplest schedule using all of the Blackwell features with minimal pipelining/asynchrony
- Excellent initial schedule for Blackwell attention optimization
- Effective for certain attention shapes
- Inefficient use of `MMA` and `MUFU` units

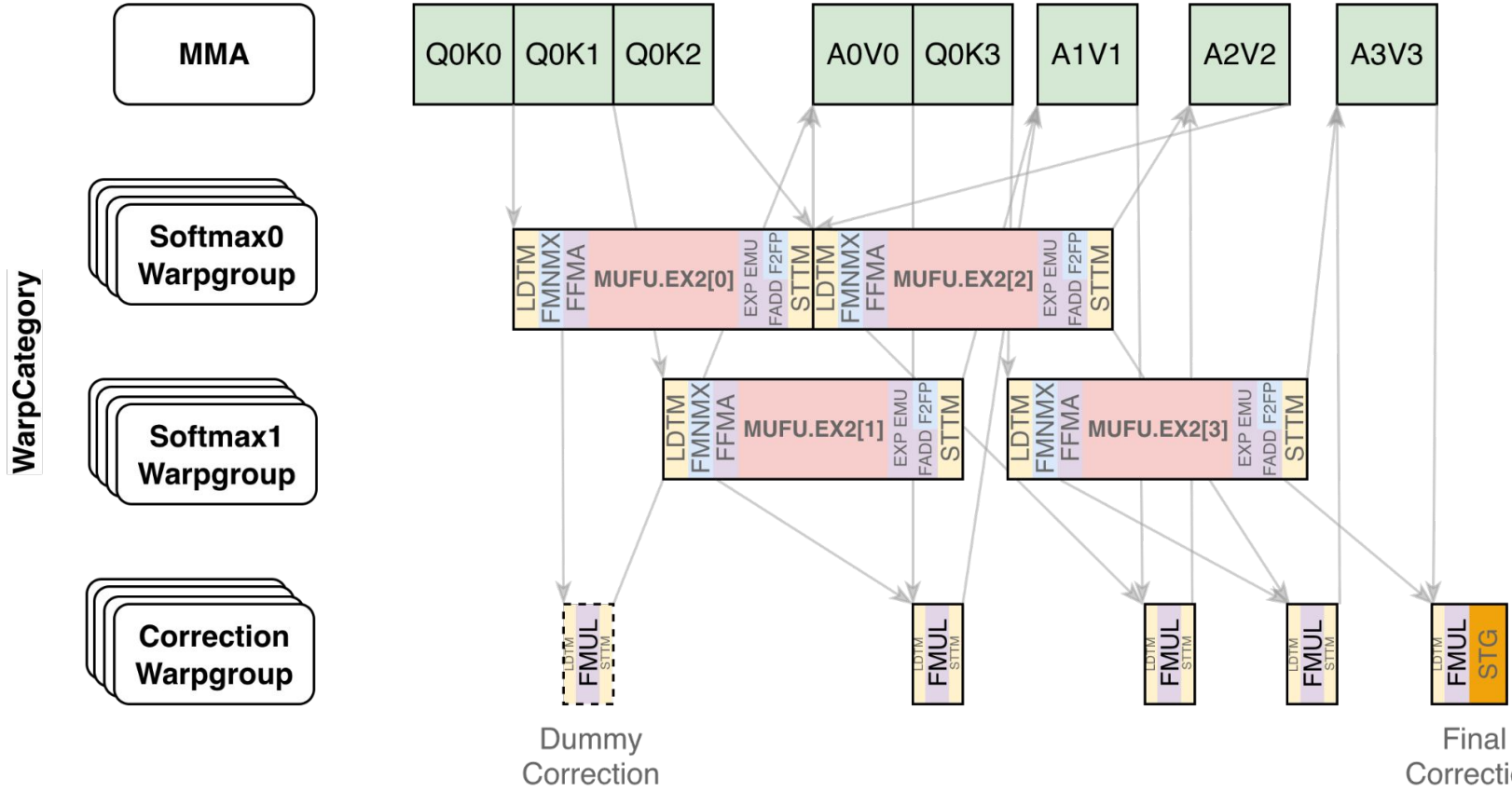
2xSoftmax

1SM loads 1 Q tile, loads 4 KV tiles, computes 4 softmax tiles with 2 softmax warpgroup



2xSoftmax - PreCompute QKs and EXP Emulation

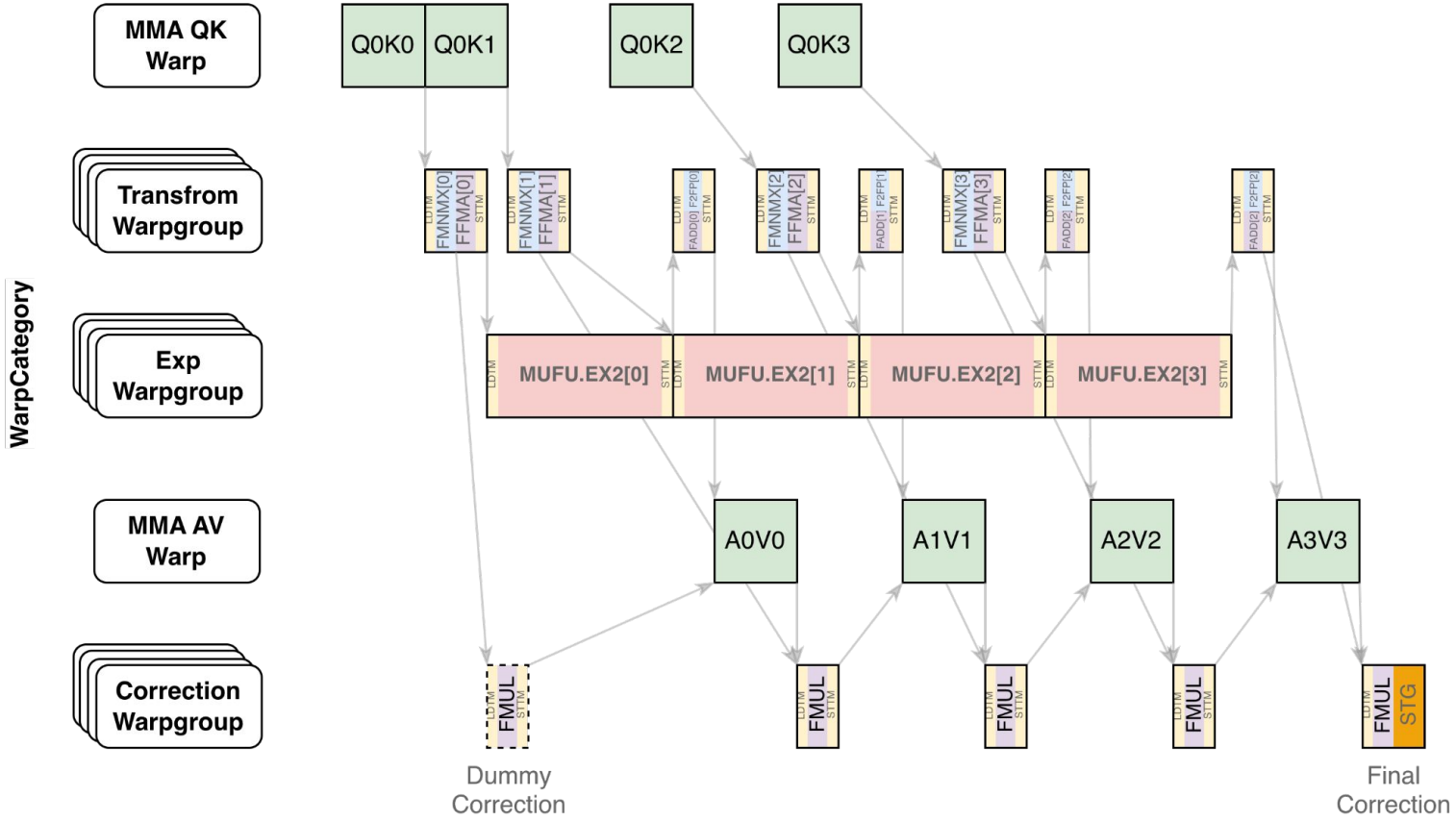
1SM loads 1 Q tile, loads 4 KV tiles, computes 4 softmax tiles with 2 softmax warpgroup



Exp-Transform

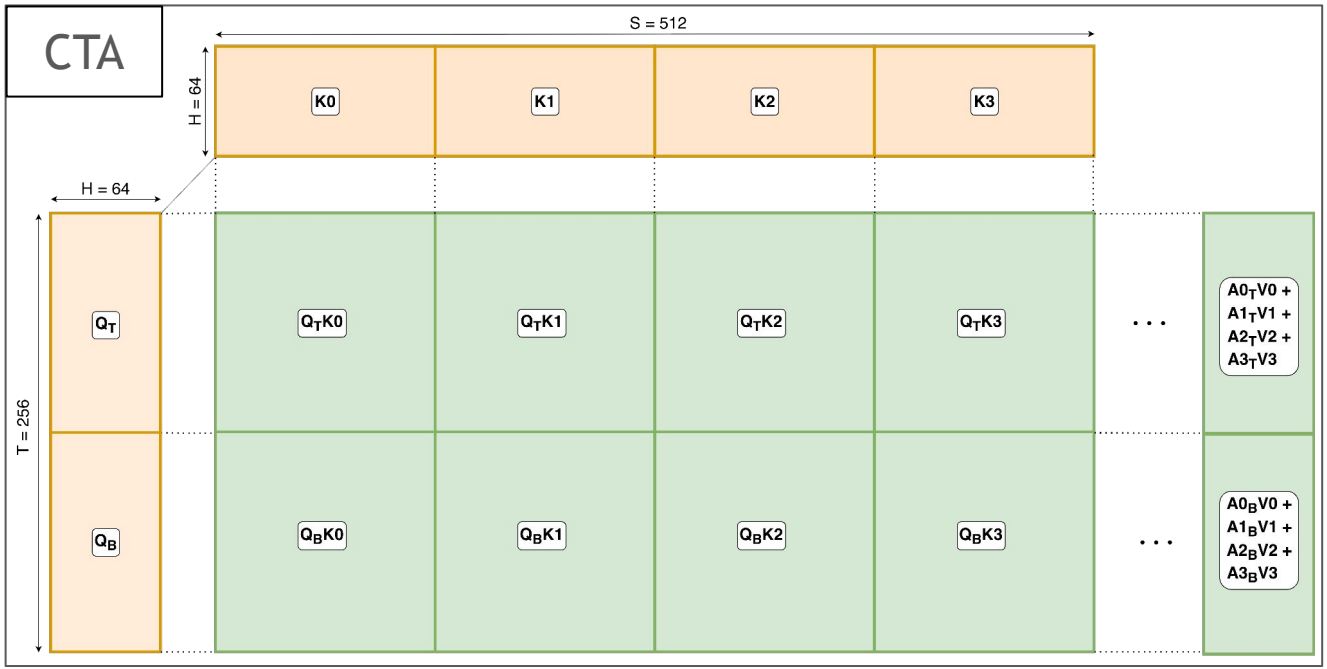
Make every functional unit feel special

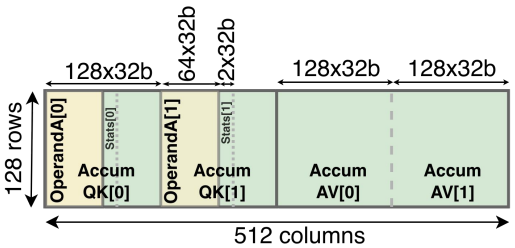
1SM loads 1 Q tile, iterates 4 KV tiles, computes 4 softmax tiles with a Transform and an Exp warpgroup



2xQ

1SM loads 2 Q tile, loads 4 KV tiles, computes 8 softmax tiles with 2 softmax warpgroups



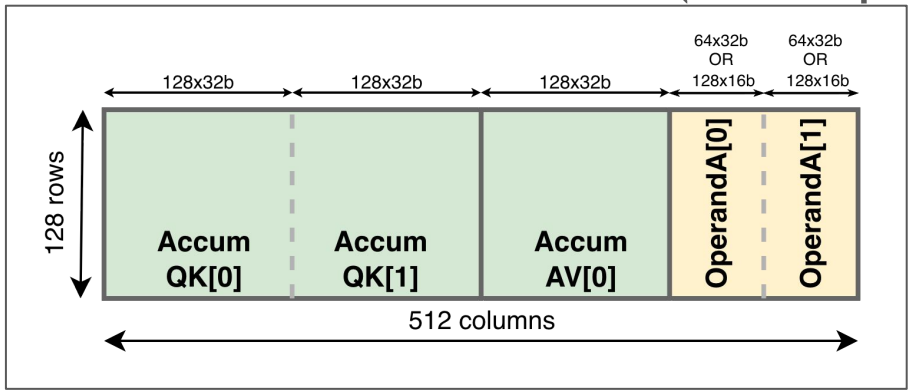


TMEM Partioniting

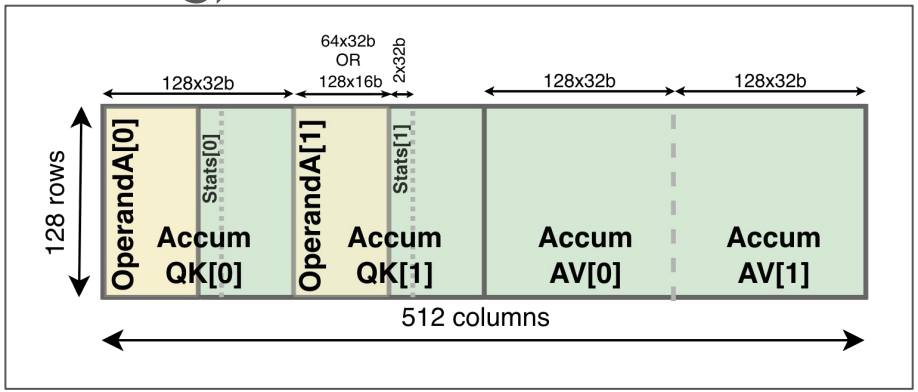
Warp-specialized Kernel Schedule first pioneered in **CUTLASS example 77, Markus Hoehnerbach** et. al. and now optimized in **FA4, Ted Zadouri** et. al.

Tensor Memory on Blackwell

(TMEM partitioning)



Non Overlapping

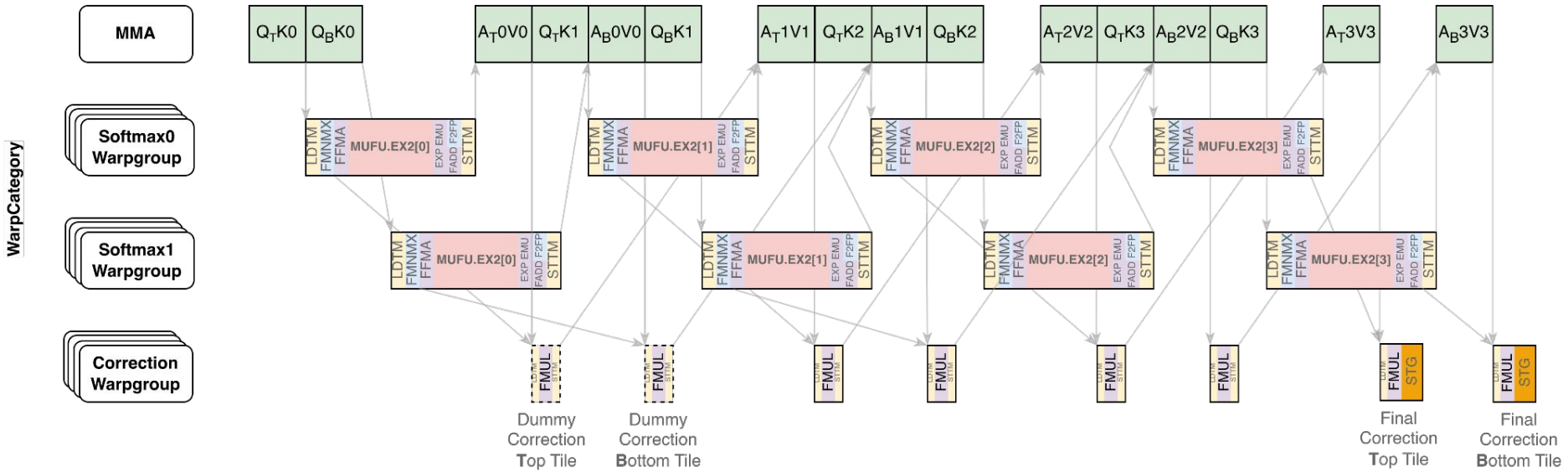


Overlapping

Non Overlapping TMEM Sections	Overlapping TMEM Sections
Dedicated TMEM slots for 2xAccumQK and 2xOperandA **Dedicated** TMEM slots for 1xAccumAV	**Overlapping** TMEM slots for 2xAccumQK and 2xOperandA **Dedicated** TMEM slots for 2xAccumAV
One slot for **AccumAV**	**Two** slots for **AccumAV**
AccumQK[i] stay in **critical region** for **shorter** duration	**Accum QK[i]** stay in **critical region** for **longer** duration

2xQ

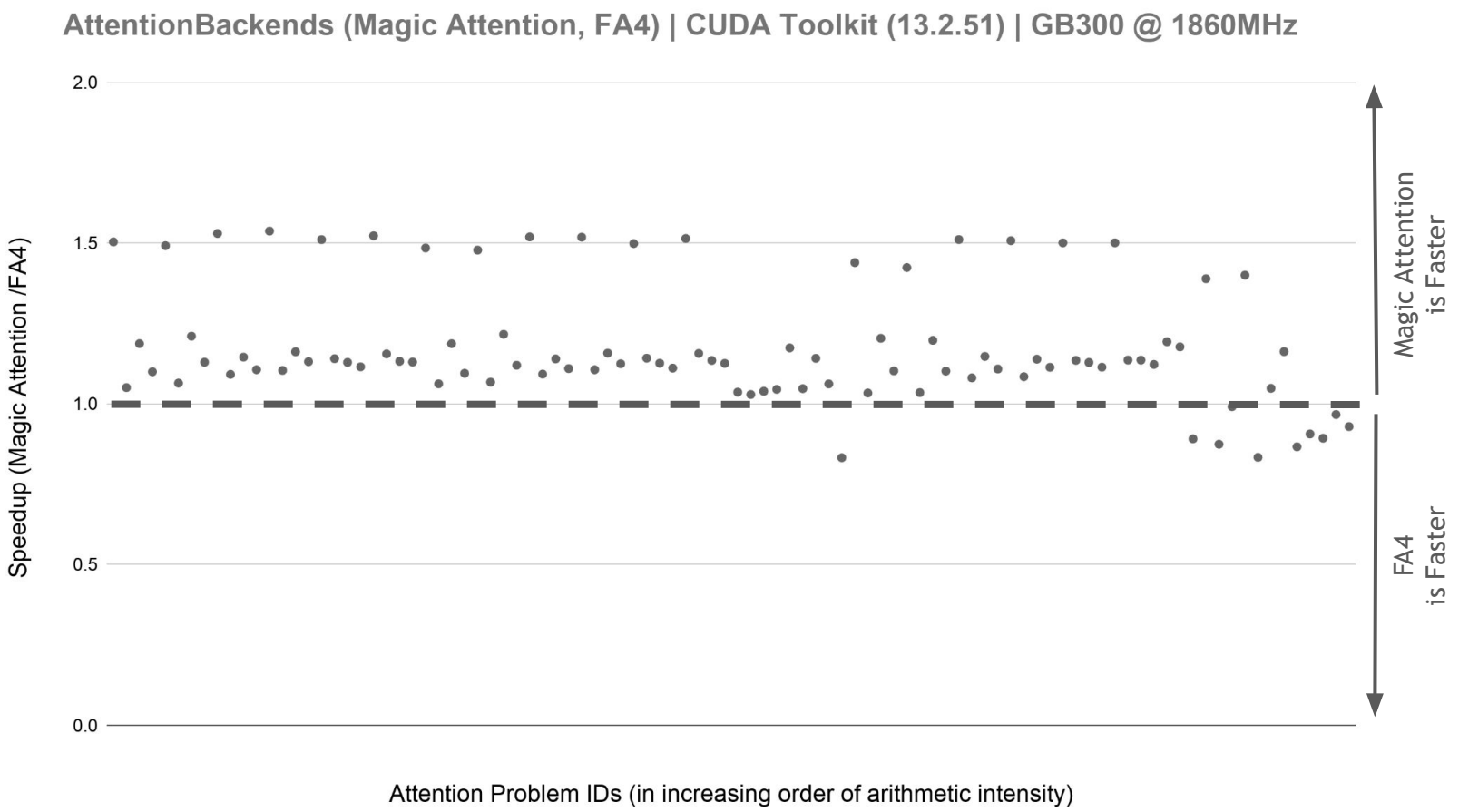
1SM loads 2 Q tile, loads 4 KV tiles, computes 8 softmax tiles with 2 softmax warpgroups



Warp-specialized Kernel Schedule first pioneered in **CUTLASS example 77, Markus Hoehnerbach** et. al. and now optimized in **FA4, Ted Zadouri** et. al.

Magic

GB300


AttentionBackends (Magic Attention, FA4) | CUDA Toolkit (13.2.51) | GB300 @ 1860MHz
Speedup (Magic Attention /FA4)
2.0
1.5
1.0
0.5
0.0
Magic Attention is Faster
FA4 is Faster
Attention Problem IDs (in increasing order of arithmetic intensity)

Magic

Thank you!

Magic